HACK THE BOX MEETUP • WORKSHOP GUIDE

Apple Silicon Setup Guide (M1 / M2 / M3 / M4)

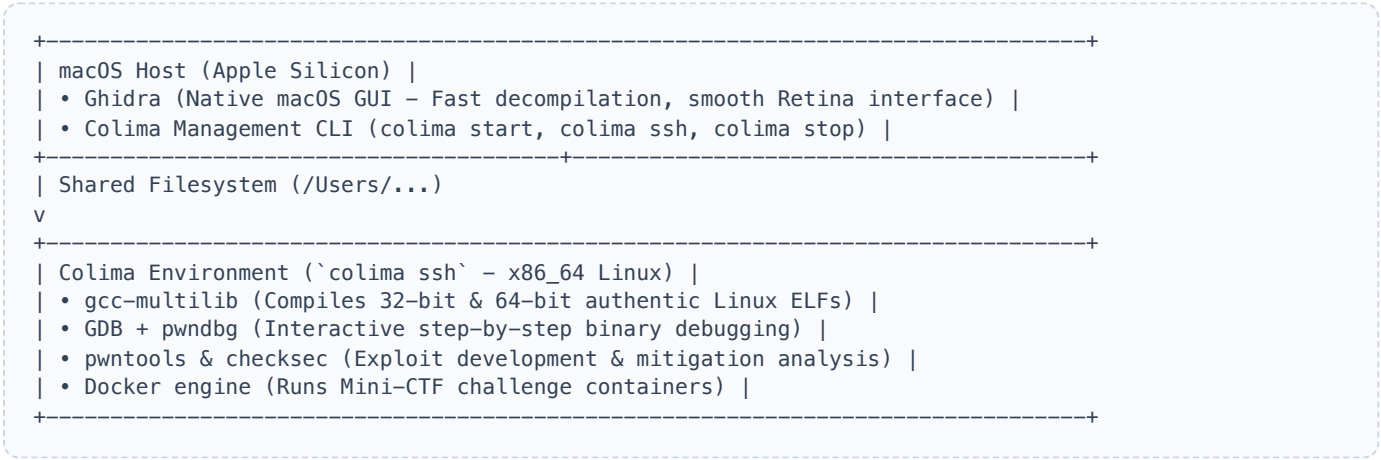
Lightweight Binary Exploitation (PWN) Setup • Host: macOS • Execution: Colima Linux (x86_64)

1. Overview & Concept

Running a full desktop Linux virtual machine (e.g. Kali Linux inside VMware Fusion or UTM) under x86_64 emulation on Apple Silicon consumes **8GB+ RAM**, **40GB+ disk space**, and suffers from frustrating cursor/GUI lag.

To solve this, we use a **Lightweight Split Architecture**:

- **macOS Host:** Only runs **Ghidra** natively with hardware acceleration and zero GUI lag.
- **Colima (QEMU x86_64):** A headless Linux VM taking only ~2GB RAM. Once you enter via `colima ssh`, you work in a **typical Kali Linux environment**—compiling with GCC, debugging with GDB + pwndbg, running CTF containers, and writing/running exploits.



Bonus: Colima automatically mounts your macOS user directory at `/Users/...`, so any file created on macOS is instantly available inside Colima, and vice versa!

2. Phase 1: macOS Host Setup (Getting Colima & Ghidra Ready)

Run these commands in your **macOS Terminal**:

Step 1: Ensure Homebrew is Installed

```
which brew || /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/insta
```

Step 2: Install Ghidra on macOS Host

```
# Install Ghidra and OpenJDK 21
brew install ghidra

# Symlink so typing 'ghidra' launches it directly from any terminal
ln -sf /opt/homebrew/bin/ghidraRun /opt/homebrew/bin/ghidra
```

> **✔ Ghidra Verification**
>
> Run `which ghidra`. It should output `/opt/homebrew/bin/ghidra`.

Step 3: Install Colima, QEMU & Lima

```
brew install colima qemu lima lima-additional-guestagents docker docker-compose
```

> ⚠ **Apple Silicon Lima Fix (Required)**
>
> On Apple Silicon, Lima only bundles the ARM64 guest agent by default. The `lima-additional-guestagents` package provides the x86_64 agent. Link it into Lima's Cellar directory:
>
> ```
> ln -sf /opt/homebrew/share/lima/lima-guestagent.*.gz /opt/homebrew/Cellar/lima/*/share/lima/
> ```

Step 4: Start Colima in x86_64 Mode

```
colima start --arch x86_64 --vm-type qemu
```

Verify that Colima is running with an x86_64 architecture:

```
colima status
# Output should show: arch: x86_64, colima is running using QEMU
```

3. Phase 2: Enter the Linux Environment (`colima ssh`)

From your macOS terminal, enter the headless x86_64 Linux machine:

```
colima ssh
```

> 🚀 **You are now inside x86_64 Linux!**
>
> Everything from this point forward is executed inside this session, just like a Kali Linux terminal.

4. Phase 3: Setup Inside Colima (Like Typical Kali Linux)

Inside your `colima ssh` terminal:

Step 1: Navigate to the Workshop Repository

```
# Your macOS files are automatically mounted under /Users/...
cd /Users/<your-username>/Documents/GitHub/htb-meetup-pwn

# (Or clone directly inside Colima if preferred):
# git clone https://github.com/danishhakim/htb-meetup-pwn.git && cd htb-meetup-pwn
```

Step 2: Install PWN Tools inside Colima

```
chmod +x scripts/install-tools.sh
./scripts/install-tools.sh --skip-ghidra
```

This automatically provisions:

- `build-essential` & `gcc-multilib` (32-bit and 64-bit C compilers)
- `gdb` & `pwndbg` (exploit development debugger with register visualization)
- `python3`, `pip3` & `pwntools`
- `checksec` (ELF binary security analyzer)
- `socat`, `netcat`, and debugging utilities

5. Phase 4: Compiling, Debugging & Testing Challenges

All hands-on compilation, GDB inspection, and solving happen directly inside Colima:

A. Guided Hands-on Challenges (`handson/`)

```
cd handson

# 1. Compile 32-bit and 64-bit vulnerable binaries
./build.sh

# 2. Inspect mitigations with checksec
checksec --file=x64/challenge
checksec --file=x86/challenge

# 3. Interactive Debugging with pwndbg
gdb ./x64/challenge
# pwndbg> r
# pwndbg> cyclic 100

# 4. Run solve scripts
cd x64 && python3 solve.py
# Output: FLAG{x64_r3t2w1n_b4s1cs}

cd ../x86 && python3 solve.py
# Output: FLAG{x86_r3t2w1n_b4s1cs}
```

B. Mini-CTF Docker Challenges (`mini-ctf/`)

```
cd /Users/<your-username>/Documents/GitHub/htb-meetup-pwn/mini-ctf

# 1. Start challenge services via Docker
docker compose up -d

# 2. Extract challenge binaries for reversing
./extract-binaries.sh

# 3. Run solve scripts against local services
python3 x64/solve.py
# Output: 🚩 HTB{x64_st4ck_sm4sh1ng_pr0}

python3 x86/solve.py
# Output: 🚩 HTB{x86_buff3r_0v3rfl0w_m4st3r}

# 4. Tear down when done
docker compose down
```

6. Phase 5: Reversing with Ghidra (macOS Host)

Whenever you need to inspect code or decompile a target binary:

1. Open a **new terminal tab on your macOS host** (outside Colima).
2. Type `ghidra` to launch the GUI.
3. Create a project and import the binary from the shared folder (e.g. `handson/x64/challenge` or `mini-ctf/binaries/challenge_x86`).
4. Analyze functions, determine buffer offsets, and locate target functions (`win()` , `get_flag()`).
5. Switch back to your `colima ssh` terminal to craft the exploit and debug in GDB!

7. Colima Lifecycle Commands (macOS Host)

Command	Where	Description
`colima ssh`	macOS	Enters the x86_64 Linux environment.
`exit`	Colima	Exits the Colima shell and returns to macOS.
`colima stop`	macOS	Pauses the Linux VM to save battery and memory.
`colima start`	macOS	Resumes the Colima VM with all installed tools and state intact.
`colima delete -f`	macOS	Removes the VM and frees all disk space when workshop is complete.

8. Troubleshooting Reference

Error / Symptom	Environment	Solution
`guest agent binary could not be found for Linux-x86_64`	macOS	Run `brew install lima-additional-guestagents` and link: `ln -sf /opt/homebrew/share/lima/lima-guestagent.*.gz /opt/homebrew/Cellar/lima/*/share/lima/`
`ghidra: command not found`	macOS	Run `ln -sf /opt/homebrew/bin/ghidraRun /opt/homebrew/bin/ghidra`
Cannot find repo in Colima	Colima	Files are mapped at identical path: `cd /Users/<your-username>/...`
GDB doesn't load pwndbg	Colima	Run `echo "source /usr/local/lib/pwndbg-gdb/gdbinit.py" >> ~/.gdbinit`
`ModuleNotFoundError: No module named 'pwn'`	Colima	Run `pip3 install pwntools --break-system-packages`